FREELANCE PORTFOLIO CLI

PROYECTO MODULO NODE JS

Jeshua Perez

Daniel Borja

CAMPUSLANDS GUATEMALA
SALON QUETZAL
CAMPUS TEC ZONA 4
CIUDAD DE GUATEMALA
2026

FREELANCE PORTFOLIO CLI

1. SITUACIÓN PROBLEMA

En el modelo de trabajo independiente (freelance), el profesional asume no solo el rol técnico de desarrollo, diseño o consultoría, sino también la totalidad de las responsabilidades administrativas, comerciales y financieras de su actividad económica. En la práctica actual, los freelancers gestionan sus flujos de trabajo utilizando herramientas dispersas (hojas de cálculo desconectadas, documentos de texto plano, notas rápidas y registros manuales).

Esto genera varias problemáticas, tales como:

1. Perdida de trazabilidad en el ciclo comercial: No existe una vinculación directa entre una propuesta enviada y la formalización de un proyecto.
2. Inconsistencia y riesgo financiero: El registro de gastos operativos, anticipos, ingresos o egresos se realiza sin controles ni validaciones.
3. Falta de control de entregables y alcance: Los cambios imprevistos dentro del proyecto pueden romper la correlación con pagos recibidos o fechas contractuales.
4. Sobrecarga cognitiva y bloqueos del desarrollador: El tiempo que se necesita para ingresar manualmente los datos y llevar un orden reduce la productividad de las tareas necesarias.

Por tanto, surge la necesidad de construir una herramienta unificada, ligera y ágil en línea de comandos (CLI) mediante Node.js, respaldada por una base de datos NoSQL (MongoDB) que implemente transacciones reales (ACID) y una arquitectura orientada a objetos desacoplada (SOLID y Patrones de Diseño). Esto garantiza atomicidad en las transacciones comerciales/financieras y un control riguroso de extremo a extremo del portafolio profesional.

2. LEVANTAMIENTO DE REQUERIMIENTOS

El levantamiento de requerimientos se llevó a cabo mediante el análisis del flujo de trabajo habitual de un trabajador autónomo, identificando los actores, entidades de dominio y reglas de consistencia de negocio requeridas:

- **Actor Principal:** Freelancer / Administrador del Portafolio (opera a través de una interfaz interactiva de consola).
- **Entidades Principales:** Clientes, Propuestas Comerciales, Proyectos, Contratos, Entregables y Movimientos Financieros (Ingresos/Egresos).
- **Mecanismos de Interacción:** Interfaz de línea de comandos basada en menús guiados (inquirer), retroalimentación visual estandarizada (chalk) y validaciones directas en capas de dominio (/models) antes de la persistencia.

3. REQUERIMIENTOS

3.1. Requerimientos Funcionales

3.1.1. GESTION CRUD DE CLIENTES

3.1.1.1. El sistema debe permitir registrar, consultar, actualizar y eliminar clientes con validación estricta de correo electrónico y datos obligatorios.

3.1.2. ASOSIACIÓN DE ENTIDADES

3.1.2.1. El sistema debe permitir vincular a un cliente específico con múltiples propuestas, proyectos y transacciones.

3.1.3. GESTIÓN DE PROPUESTAS

3.1.3.1. El sistema debe registrar propuestas con descripción, precio, plazos de entrega y estado (pendiente, aceptada, rechazada).

3.1.4. CONVERSION AUTOMATICA

3.1.4.1. Al cambiar el estado de una propuesta a aceptada, el sistema debe generar automáticamente un nuevo proyecto vinculado al cliente de dicha propuesta.

3.1.5. CONTROL DE CICLO DE VIDA

3.1.5.1. El sistema debe permitir actualizar los estados del proyecto (activo, pausado, finalizado, cancelado) y registrar el avance porcentual (0 - 100%).

3.1.6. GENERACIÓN Y ASOCIACIÓN CONTRACTUAL

3.1.6.1. El sistema debe asociar un contrato a cada proyecto, definiendo términos y condiciones, fecha de inicio, fecha estimada de fin y valor total pactado.

3.1.7. GESTION DE ENTREGABLES POR HITO

3.1.7.1. El sistema debe permitir agregar entregables por proyecto con fecha límite y estado (pendiente, entregado, aprobado, rechazado).

3.1.8. ROLLBACK Y CONSISTENCIA

3.1.8.1. El sistema debe revertir cambios de estado o eliminaciones de entregables mediante transacciones seguras si una operación complementaria falla.

3.1.9. REGISTRO DE INGRESOS Y EGRESOS

3.1.9.1. El sistema debe registrar entradas (pagos de clientes) y salidas (gastos asociados) vinculados a un proyecto específico.

3.1.10. CALCULO DE BALANCES

3.1.10.1. El sistema debe calcular y mostrar el balance neto global, filtrado por cliente o por rango de fechas.

3.1.11. TRANSACCIONALIDAD DE PAGOS

3.1.11.1. El registro de pagos y la actualización del estado de cuentas/entregables deben ejecutarse dentro de una transacción atómica para prevenir duplicidades o inconsistencias.

3.2. Requerimientos No Funcionales

3.2.1. PROGRAMACIÓN ORIENTADA A OBJETOS Y SOLID

3.2.1.1. Todo el código base debe estar estructurado en clases y módulos desacoplados bajo principios SOLID (Single Responsibility, Dependency Inversion, etc.).

3.2.2. PATRONES DE DISEÑO DE SOFTWARE

3.2.2.1. Implementación obligatoria de al menos dos patrones reconocidos: Repository Pattern (para desacoplar persistencia) y Command/Factory Pattern (para el control de la CLI y creación de modelos).

3.2.3. NATIVO DE MONGODB

3.2.3.1. La base de datos debe gestionarse exclusivamente con el driver oficial mongodb para Node.js, prescindiendo expresamente de librerías ODM de alto nivel como mongoose.

3.2.4. TRANSACCIONES ACID

3.2.4.1. Las operaciones compuestas (aprobación de propuestas, pagos y entregables) deben implementar session.startTransaction(), commitTransaction() y abortTransaction().

3.2.5. EXPERIENCIA UI DE CONSOLA

3.2.5.1. La navegación debe ser interactiva y amigable mediante inquirer, con mensajes de estado y errores estilizados con chalk.

3.2.6. VALIDACION DE DATOS

3.2.6.1. La carpeta /models debe validar tipos, longitudes, rangos y campos obligatorios antes de enviar los documentos al repositorio de base de datos.

3.2.7. GESTION SEGURA DE VARIABLES DE ENTORNO

3.2.7.1. Las cadenas de conexión, nombres de bases de datos y configuraciones sensibles deben cargarse mediante variables de entorno usando dotenv.

3.2.8. ESTANDAR DE CONTROL DE VERSIONES

3.2.8.1. El repositorio debe documentar el flujo de desarrollo utilizando la especificación de Conventional Commits (feat:, fix:, docs:, refactor:).

4. HISTORIAS DE USUARIO CON CRITERIOS DE ACEPTACIÓN

1.

HISTORIA DE USUARIO			
Prioridad: Alta			
Código de Requerimiento:	1.1.1.1	**Actor**	Freelancer
Nombre de Requerimiento:	**GESTION CRUD DE CLIENTES**		
Descripción			
Como freelancer, quiero registrar, consultar, actualizar y eliminar clientes en el sistema mediante la consola, para mantener mi directorio de contactos comerciales organizado.			
Funcionalidad			
Permite crear nuevos clientes, listar todos los existentes, editar información de contacto y dar de baja registros que no posean dependencias críticas.			
Criterios de aceptación	1. El formulario de consola solicita nombre, correo, teléfono y empresa mediante inquirer. 2. El modelo valida que el nombre sea obligatorio y que el correo tenga un formato electrónico válido antes de guardar. 3. La persistencia se realiza en la colección clients de MongoDB usando el driver nativo. 4. La CLI muestra mensajes de éxito en color verde y alertas de error en color rojo usando chalk.		
Restricciones			
No usar Mongoose; la validación de campos obligatorios y formato debe ejecutarse dentro de la clase de dominio en /models.			

2.

HISTORIA DE USUARIO			
Prioridad: Media			
Código de Requerimiento:	1.1.2.1	**Actor**	Freelancer
Nombre de Requerimiento:	**Asociación de Entidades al Cliente**		
Descripción			
Como freelancer, quiero consultar el perfil de un cliente y visualizar todas sus propuestas, proyectos y finanzas relacionadas, para conocer el historial completo de mi relación comercial.			
Funcionalidad			

Permite seleccionar un cliente y generar una vista consolidada de todas las entidades asociadas a su identificador único.	
Criterios de aceptación	1. La CLI lista los clientes registrados para seleccionar uno por su identificador. 2. El repositorio realiza las consultas referenciales hacia las colecciones de propuestas, proyectos y transacciones. 3. Se presenta la información agrupada por secciones de forma clara en la terminal.
Restricciones	
El cliente seleccionado debe existir en MongoDB; las referencias deben validarse como ObjectId válidos.	

3.

HISTORIA DE USUARIO			
Prioridad: Alta			
Código de Requerimiento:	1.1.3.1	**Actor**	Freelancer
Nombre de Requerimiento:	**Creación y Listado de Propuestas**		
Descripción			
Como freelancer, quiero crear propuestas comerciales definiendo descripción, precio, plazos y estado, para gestionar mis cotizaciones pendientes.			
Funcionalidad			
Registro de propuestas comerciales vinculadas a un cliente con asignación de estados iniciales y plazos de entrega.			
Criterios de aceptación	1. Permite seleccionar un cliente existente de la lista interactiva. 2. Valida que el precio sea un número estrictamente mayor a 0 y que el plazo en días sea un entero positivo. 3. El estado inicial de la propuesta se asigna automáticamente como pendiente. 4. Permite filtrar el listado de propuestas por estado (pendiente, aceptada, rechazada).		
Restricciones			
Solo se admiten los estados predefinidos en el modelo (pendiente, aceptada, rechazada).			

4.

HISTORIA DE USUARIO			
Prioridad: Alta			
Código de Requerimiento:	1.1.4.1	**Actor**	Freelancer
Nombre de Requerimiento:	**Conversión Automática de Propuesta a Proyecto**		
Descripción			
Como freelancer, quiero que al marcar una propuesta como aceptada se cree automáticamente su proyecto en el sistema, para iniciar la ejecución del trabajo sin duplicar información manual.			
Funcionalidad			
Cambio de estado de propuesta a aceptada y disparo automático de la instanciación de un nuevo proyecto vinculado al cliente de la propuesta.			
Criterios de aceptación	1. La operación se ejecuta mediante una transacción real en MongoDB con session.startTransaction(). 2. Se actualiza el estado de la propuesta a aceptada y se inserta el documento en la colección projects. 3. El proyecto creado inicia con estado activo y avance en 0%. 4. Si la creación del proyecto falla, la propuesta no modifica su estado original (abortTransaction).		
Restricciones			
Uso estricto de transacciones ACID en el driver oficial de MongoDB.			

5.

HISTORIA DE USUARIO			
Prioridad: Media			
Código de Requerimiento:	1.1.5.1	**Actor**	Freelancer
Nombre de Requerimiento:	**Control del Ciclo de Vida del Proyecto**		
Descripción			
Como freelancer, quiero cambiar el estado de mis proyectos y actualizar su porcentaje de avance, para monitorear el progreso de mis trabajos activos.			
Funcionalidad			
Menú de actualización para proyectos donde se modifica el estado operativo y el porcentaje de avance acumulado.			
Criterios de aceptación	1. Permite seleccionar estados válidos: activo,		

	pausado, finalizado o cancelado. 2. El campo de avance valida que el número ingresado esté contenido en el rango de 0 a 100%. 3. Se confirma la actualización en consola mostrando el nuevo avance formateado visualmente.
Restricciones	
No permitir avances negativos ni superiores a 100%.	

6.

HISTORIA DE USUARIO			
Prioridad: Alta			
Código de Requerimiento:	1.1.6.1	**Actor**	Freelancer
Nombre de Requerimiento:	**Generación y Vinculación de Contrato**		
Descripción			
Como freelancer, quiero registrar un contrato asociado a un proyecto con sus cláusulas, fechas y valor total, para respaldar legal y comercialmente el trabajo.			
Funcionalidad			
Módulo de creación y consulta de acuerdos contractuales ligados a un proyecto activo.			
Criterios de aceptación	1. Solicita condiciones generales, fecha de inicio, fecha de fin y valor económico acordado. 2. El modelo valida que la fecha de fin sea cronológicamente posterior a la fecha de inicio. 3. El valor total pactado debe ser un número positivo. 4. El contrato queda asociado unívocamente al ID del proyecto en MongoDB.		
Restricciones			
Un proyecto no puede tener contratos con valores totales menores o iguales a cero ni inconsistencias en el orden de fechas.			

7.

HISTORIA DE USUARIO			
Prioridad: Alta			
Código de Requerimiento:	1.1.7.1	**Actor**	Freelancer

Nombre de Requerimiento:	**Gestión de Entregables por Hito**
Descripción	
Como freelancer, quiero registrar entregables con fecha límite y cambiar sus estados de aprobación, para controlar el avance de cada hito del proyecto.	
Funcionalidad	
Creación, consulta y actualización de estados para entregables vinculados a proyectos específicos.	
Criterios de aceptación	1. Permite crear entregables ingresando título, descripción y fecha límite vinculados al ID de un proyecto. 2. Permite actualizar el estado del entregable seleccionando entre pendiente, entregado, aprobado y rechazado. 3. Valida que los textos descriptivos y de título no queden vacíos.
Restricciones	
Solo se admiten estados dentro del conjunto permitido (pendiente, entregado, aprobado, rechazado).	

8.

HISTORIA DE USUARIO			
Prioridad: Alta			
Código de Requerimiento:	1.1.8.1	**Actor**	Freelancer
Nombre de Requerimiento:	**Rollback y Consistencia en Entregables**		
Descripción			
Como freelancer, quiero que las modificaciones críticas o eliminaciones de entregables se ejecuten bajo transacciones seguras con capacidad de rollback, para prevenir inconsistencias de datos.			
Funcionalidad			
Ejecución de operaciones compuestas sobre entregables con reversión automática ante excepciones en tiempo de ejecución.			
Criterios de aceptación	1. Se abre una sesión y transacción en MongoDB antes de modificar o remover el entregable. 2. Ante cualquier error durante el proceso, se ejecuta abortTransaction() restaurando el estado previo de los datos. 3. Si la operación concluye satisfactoriamente, se aplica commitTransaction() y se notifica al usuario en la CLI.		

Restricciones
No ejecutar operaciones directas sin sesión de base de datos (client.startSession()).

9.

HISTORIA DE USUARIO			
Prioridad: Alta			
Código de Requerimiento:	1.1.9.1	**Actor**	Freelancer
Nombre de Requerimiento:	**Registro de Ingresos y Egresos**		
Descripción			
Como freelancer, quiero registrar entradas de dinero por cobros y salidas por gastos asociadas a mis proyectos, para mantener la contabilidad operativa en orden.			
Funcionalidad			
Formulario interactivo para asentar movimientos contables de tipo ingreso o egreso clasificados por categoría y vinculados a proyectos y clientes.			
Criterios de aceptación	1. Permite seleccionar el tipo de transacción: ingreso o egreso. 2. Solicita monto, categoría, concepto y proyecto vinculado. 3. Valida que el monto ingresado sea un valor numérico estrictamente positivo. 4. Almacena la fecha de registro y las referencias correspondientes en la colección transactions.		
Restricciones			
El monto no puede ser cero ni negativo; el tipo de movimiento debe ser exclusivamente ingreso o egreso.			

10.

HISTORIA DE USUARIO			
Prioridad: Media			
Código de Requerimiento:	1.1.10.1	**Actor**	Freelancer
Nombre de Requerimiento:	**Cálculo y Reporte de Balances Financieros**		
Descripción			
Como freelancer, quiero consultar el balance neto total, filtrado por cliente o por rango de fechas, para evaluar el rendimiento financiero de mi trabajo.			
Funcionalidad			
Procesamiento y consolidación de movimientos financieros para presentar reportes de ingresos, egresos y balance neto en consola.			

Criterios de aceptación	1. Ofrece menú de opciones para balance global, por cliente específico o por rango de fechas. 2. Aplica la fórmula: Total Ingresos − Total Egresos = Balance Neto. 3. Imprime en terminal un resumen formateado mostrando montos positivos en verde y saldos negativos o de déficit en rojo.
Restricciones	
En los filtros de fecha, la fecha final no puede ser anterior a la fecha inicial.	

11.

HISTORIA DE USUARIO			
Prioridad: Alta			
Código de Requerimiento:	1.1.11.1	**Actor**	Freelancer
Nombre de Requerimiento:	**Transaccionalidad en el Registro de Pagos**		
Descripción			
Como freelancer, quiero que el registro de un pago y la actualización del estado del entregable o cuenta se procesen en una única transacción atómica, para evitar duplicidades o descuadres financieros.			
Funcionalidad			
Procesamiento simultáneo y atómico de la recepción de pago monetario y el cambio de estado del entregable/proyecto asociado.			
Criterios de aceptación	1. El servicio inicia una transacción con session.startTransaction(). 2. Inserta el movimiento en transactions y actualiza el estado correspondiente en deliverables o projects dentro de la misma sesión. 3. Solo si ambas escrituras son exitosas se ejecuta session.commitTransaction(). 4. Si cualquiera de los pasos falla, se llama a session.abortTransaction(), cancelando todas las mutaciones realizadas.		
Restricciones			
Obligatorio uso de sesiones transaccionales del driver oficial mongodb; garantizar el cierre de la sesión con session.endSession() en el bloque finally.			

5. METODOLOGÍA

Para el desarrollo del proyecto FreelanceHub CLI, se adoptó el marco de trabajo ágil SCRUM, complementado con buenas prácticas de ingeniería de software orientada a objetos (POO), principios SOLID y el estándar de flujo de trabajo colaborativo GitFlow.

5.1. Marco de Trabajo Ágil (SCRUM)

El ciclo de desarrollo se planificó en iteraciones cortas (Sprints) para asegurar incrementos de software funcionales y testeables:

- Sprint Planning: Definición del objetivo de cada iteración, descomposición de los requerimientos en Historias de Usuario y estimación del esfuerzo de desarrollo.
- Daily Stand-Up Meetings: Sincronizaciones cortas para responder: ¿Qué se hizo?, ¿Qué se hará hoy? y ¿Qué impedimentos existen?
- Sprint Review & Demo: Verificación de la funcionalidad de consola (CLI) y la persistencia transaccional en MongoDB.
- Sprint Retrospective: Análisis de aciertos, cuellos de botella y compromisos de mejora en la arquitectura y flujo de Git.

5.2. Roles del Equipo

Rol SCRUM	Integrante	Responsabilidades
Product Owner (PO)	Jeshua Perez	Define las prioridades del backlog, valida el alcance funcional del portafolio y los criterios de aceptación.
Scrum Master (SM)	Daniel Borja	Facilita las ceremonias, supervisa el cumplimiento del tablero (Trello/GitHub Projects) y remueve impedimentos técnicos.
Development Team	Jeshua Perez + Daniel Borja	Diseña modelos, implementa conexiones a MongoDB, aplica patrones de diseño (SOLID) y crea la CLI.

5.3. Flujo de Trabajo

Estrategia de Ramas: Se utilizó la rama base main (código en producción), dev (integración continua) y ramas específicas por funcionalidad (feat/models-validation, feat/clients-crud, feat/finance-transactions).

Estándar de Commits: Se aplicó estrictamente la convención de Conventional Commits:

- feat: para nuevas funcionalidades.
- fix: para corrección de bugs o errores de validación.
- docs: para cambios en README o documentación.
- refactor: para mejoras de código sin alterar su comportamiento.

6. EVIDENCIA DE PLANTEAMIENTO DE PLATAFORMA DE TRABAJO

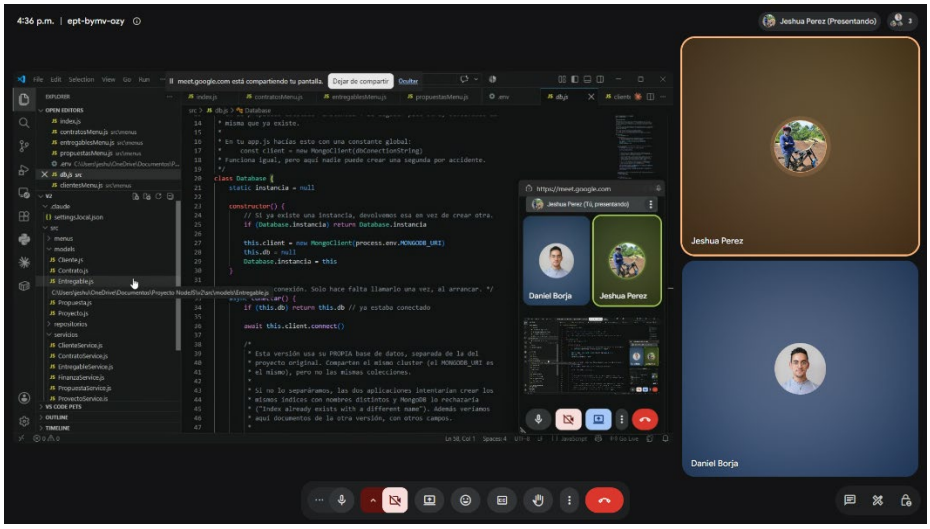
6.1. Enlaces de Proyecto

6.1.1. Repositorio de Github: https://github.com/JeshuaPerez/gestor-de-portafolio-freelance-.git

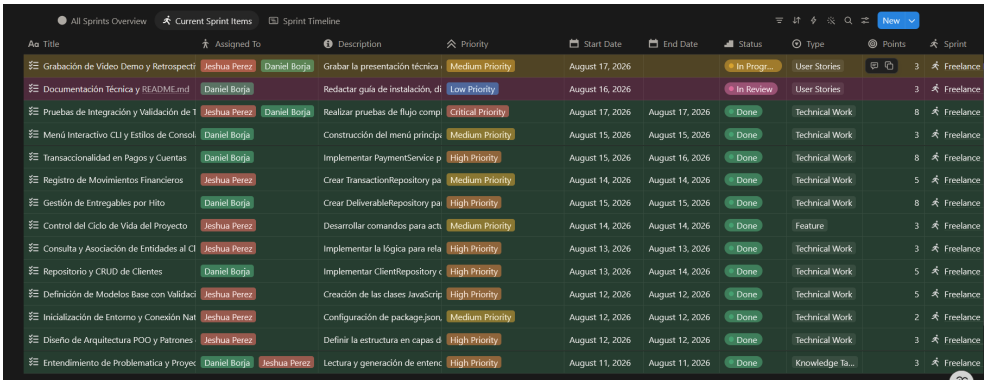
6.1.2. Video de Presentación de Producto: https://drive.google.com/drive/folders/1_GrZRgvlrmKPQLMRQV-TvCahsIXme-wQ?usp=drive_link

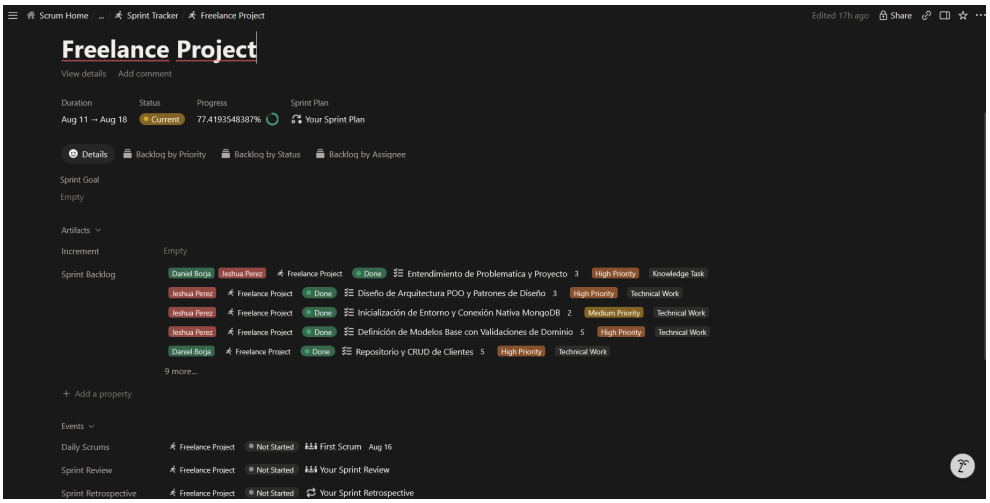
6.1.3. Link de Tablero SCRUM: https://app.notion.com/p/Scrum-Home-64c8b7658db18205aa4f01ed16e59f2a?source=copy_link

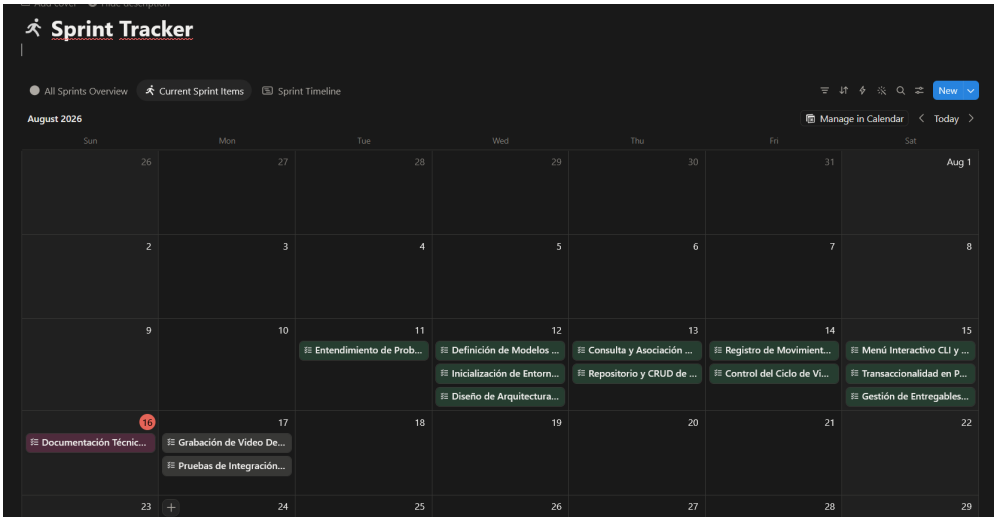
6.2. Reuniones de Seguimiento:



6.3. Evidencia de Tablero SCRUM:







6.4. Documentación de Resultados y Funcionamiento:

6.4.1. Node.js (POO): Clases estructuradas respetando Single Responsibility e Inversión de Dependencias.

6.4.2. MongoDB Driver: Comunicación directa con la base de datos sin ODM (Mongoose), permitiendo el control fino de sesiones y transacciones con session.startTransaction().

6.4.3. Inquirer & Chalk: Creación de menús dinámicos interactivos, tablas de resultados y validaciones visuales en tiempo real.

6.4.4. Dotenv: Carga segura de variables de entorno para la URI de MongoDB.

6.5. Cumplimiento de Módulos:

6.5.1. **Gestión Integral de Clientes:** CRUD completo con validación estricta de datos.

6.5.2. **Propuestas y Conversión a Proyectos:** Generación automática y atómica de proyectos al aceptar una propuesta.

6.5.3. **Entregables y Rollbacks:** Capacidad de revertir estados y transacciones ante fallos en operaciones compuestas.

6.5.4. **Módulo Financiero Transaccional:** Registro seguro de ingresos y egresos, impidiendo duplicidad de saldos y permitiendo el cálculo dinámico de balance por cliente y por fecha.

7. CONCLUSIONES

7.1. Conclusiones Generales del Documento y Proyecto

7.1.1. La implementación de una arquitectura en capas fundamentada en POO, principios SOLID y el Patrón Repository demostró ser fundamental para mantener el código desacoplado y escalable, facilitando la separación entre la interfaz de usuario en consola y la persistencia en MongoDB.

7.1.2. El uso directo del driver oficial de MongoDB, en combinación con el manejo de transacciones ACID (session.startTransaction()), garantizó la integridad referencial y financiera del sistema en procesos críticos como la conversión de propuestas a proyectos y la liquidación de entregables.

7.1.3. La aplicación de la convención de Conventional Commits y una estructura de ramas clara facilitó la trazabilidad del código y la colaboración estructurada a lo largo de todas las etapas de desarrollo.

7.2. Conclusiones de la Reunión de Retrospectiva del Sprint

7.2.1. Lo que funcionó bien: La definición temprana de las validaciones de dominio en /models redujo drásticamente la cantidad de errores al momento de interactuar con la base de datos y construir los formularios de inquirer.

7.2.2. Oportunidades de mejora detectadas: El manejo de transacciones con el driver nativo de MongoDB requirió una curva de aprendizaje inicial respecto al control de errores y cierre de sesiones (abortTransaction y

endSession), lo cual se resolvió encapsulando la lógica transaccional dentro de la capa /services.

7.2.3. Optimización futura: Implementar pruebas unitarias automatizadas para los modelos y repositorios antes de la integración con la interfaz de consola, agilizando la fase de validación de negocio.